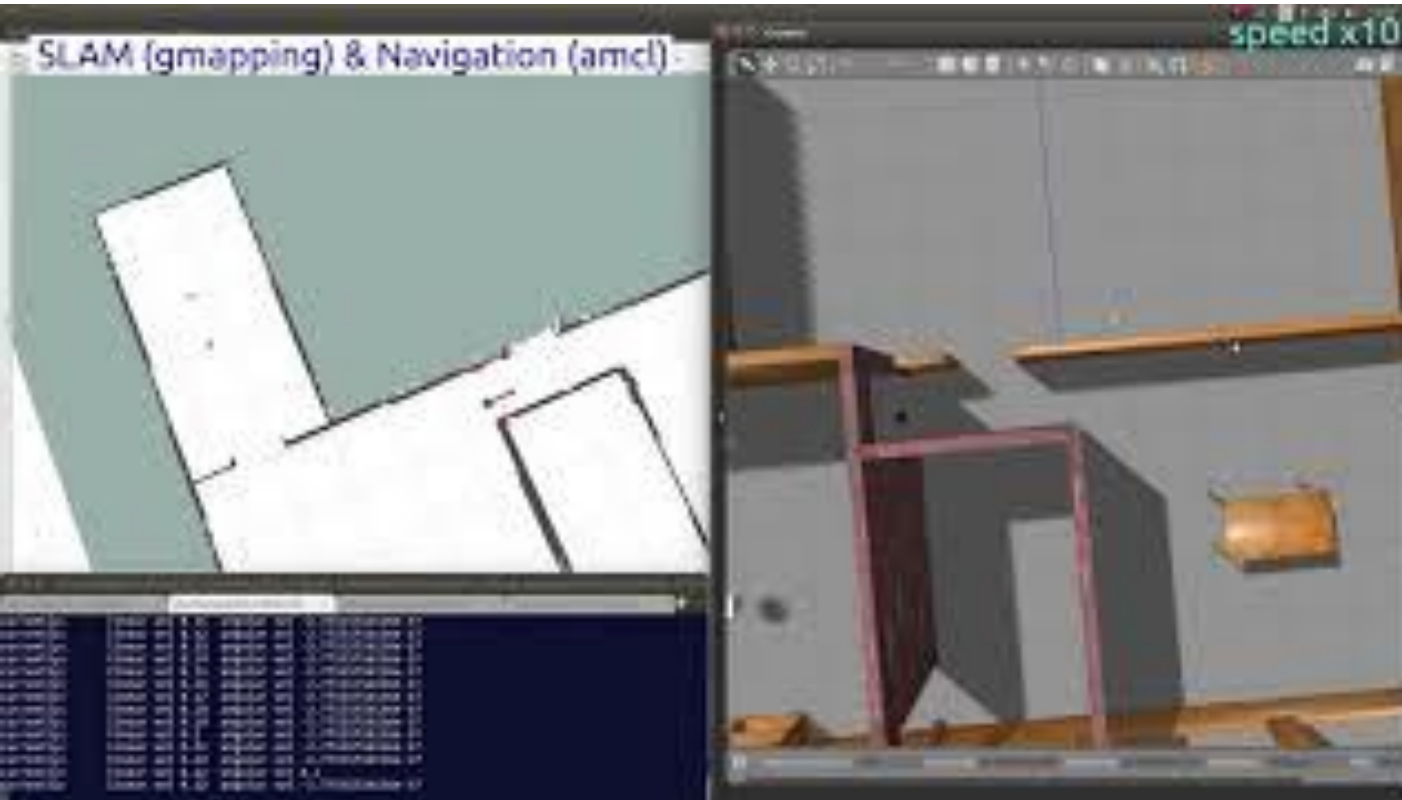


Mobile Robotics



Bachelor's
Degree in
Robotic
Intelligence

SLAM:
Simultaneous
Localization
and Mapping

SLAM

Given:

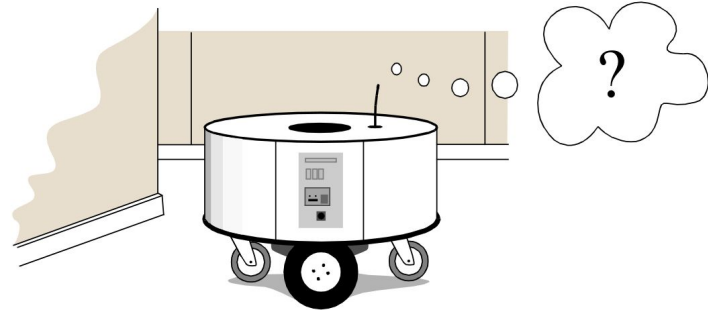
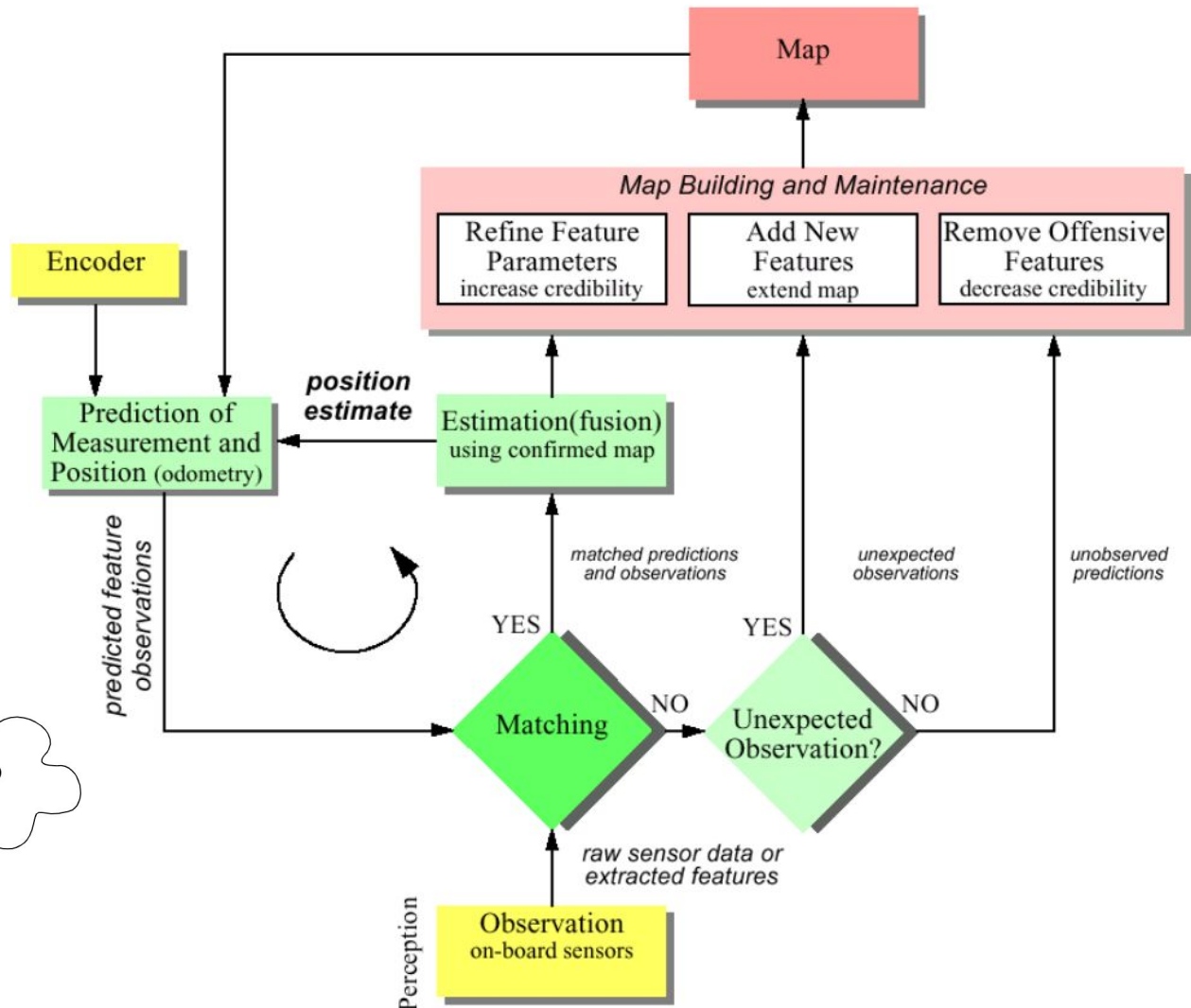
- a series of controls and
- sensor observations over discrete time steps,

the SLAM problem is to compute:

- an estimate of the agent's state and
- a map of the environment.

All quantities are usually probabilistic.

General schematic
for concurrent localization
and map building
[Siegwart et al., 2011]

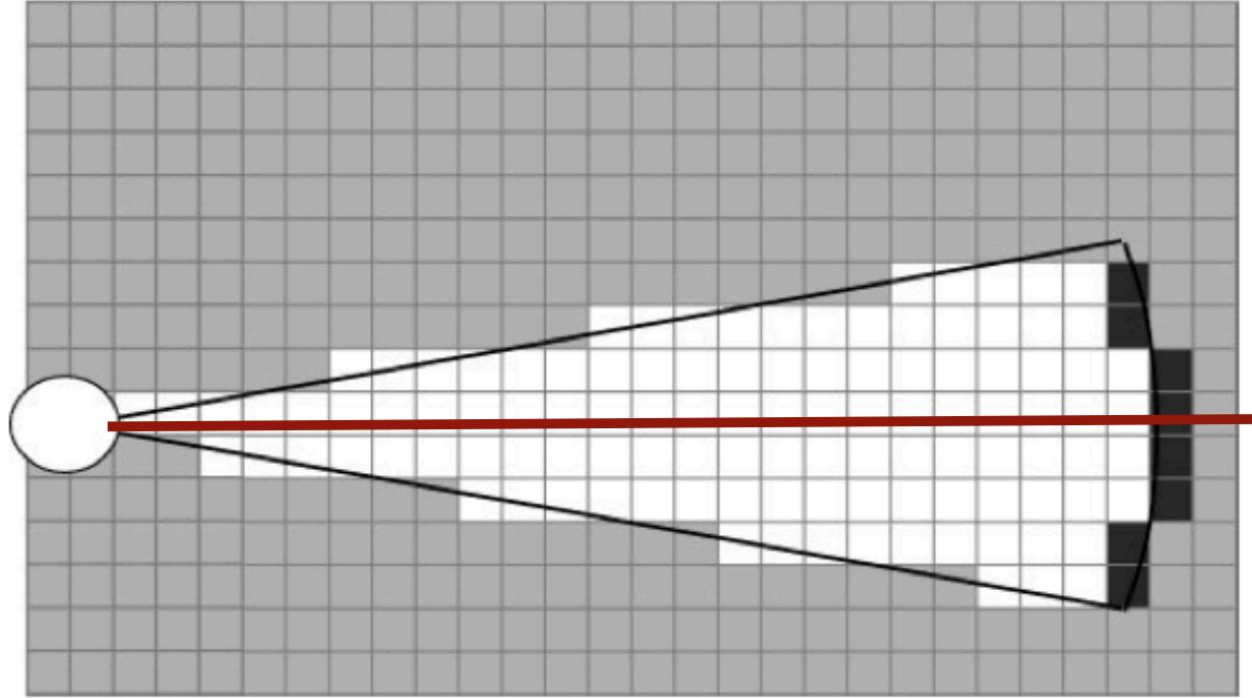


Occupancy grid map

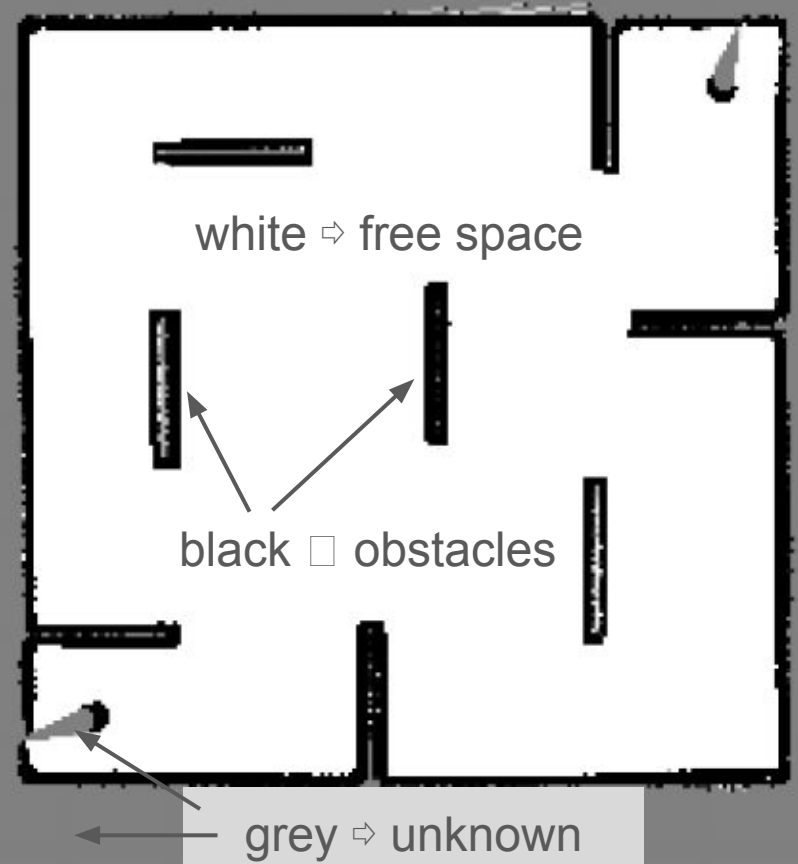
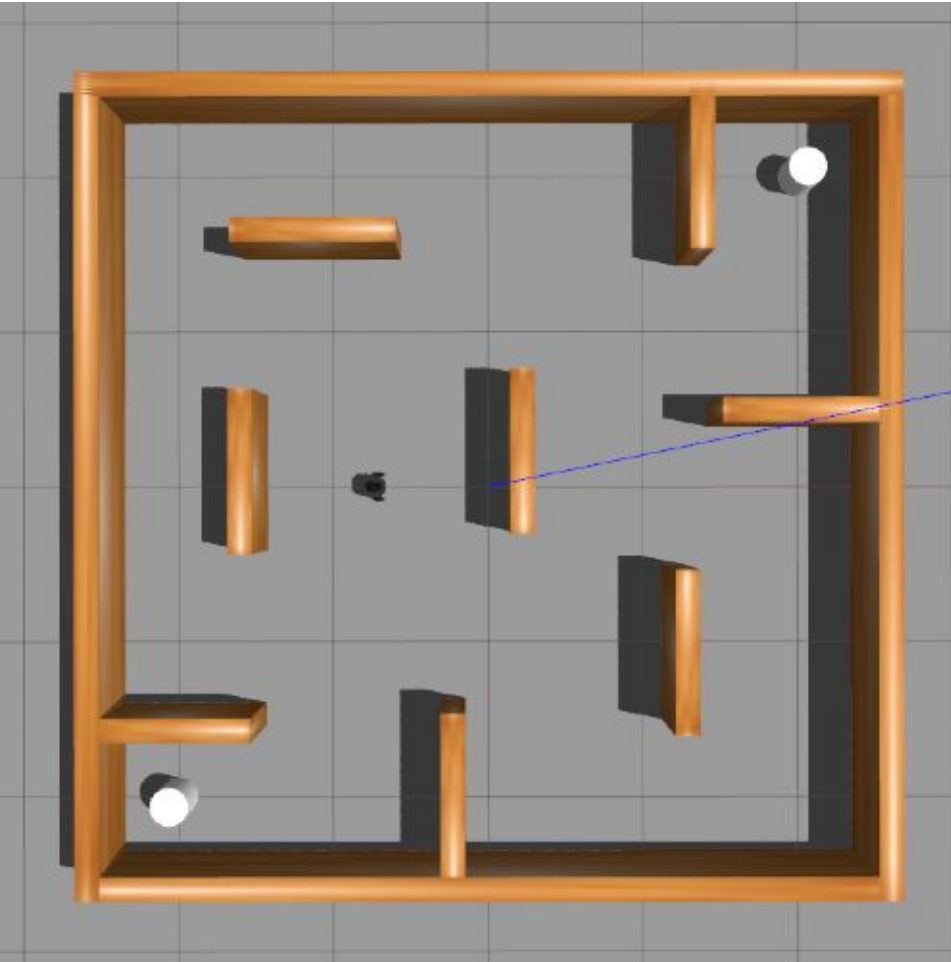
Occupancy grids are used to represent a robot workspace as a discrete grid.

Information about the environment can be collected from sensors in real time or be loaded from prior knowledge.

Laser range finders, bump sensors, cameras, and depth sensors are commonly used to find obstacles in your robot's environment.



Occupancy grid map



SLAM with TurtleBot 3 in Gazebo

Prerequisites:

```
sudo apt install ros-humble-turtlebot3-cartographer
```

Terminal 1:

```
source /opt/ros/humble/setup.bash
export ROS_LOCALHOST_ONLY=1
export TURTLEBOT3_MODEL=burger
ros2 launch turtlebot3_gazebo turtlebot3_house.launch.py
```

SLAM with TurtleBot 3 in Gazebo

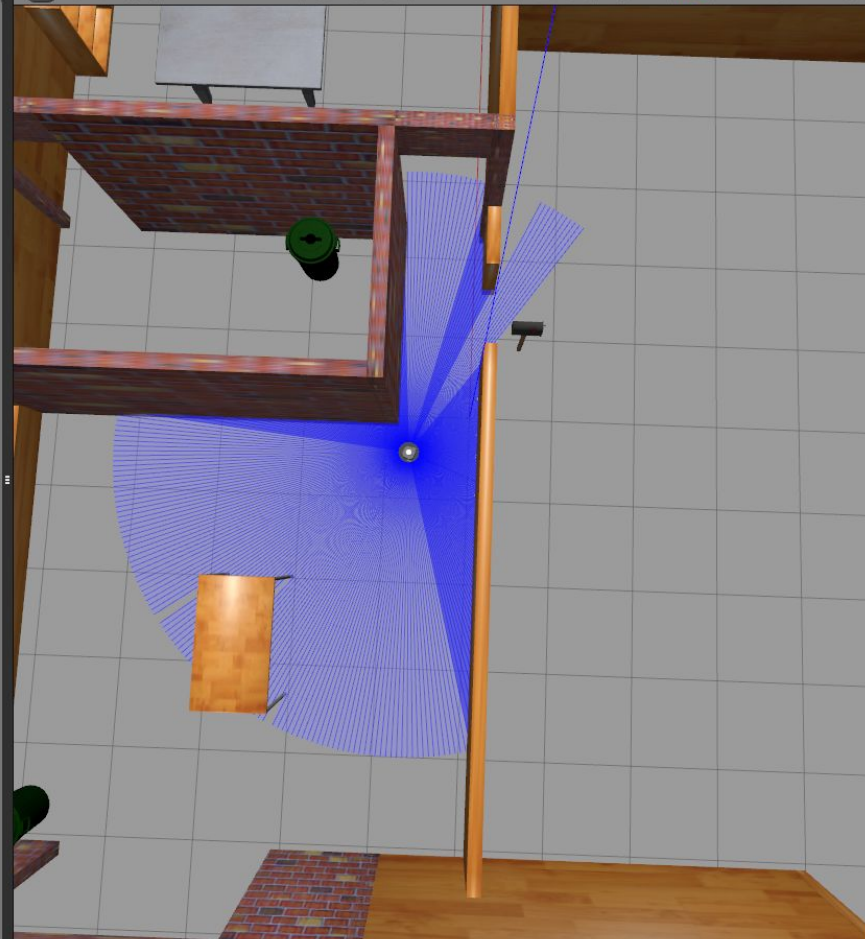
Terminal 2:

```
source /opt/ros/humble/setup.bash
export ROS_LOCALHOST_ONLY=1
export TURTLEBOT3_MODEL=burger
ros2 run turtlebot3_teleop teleop_keyboard
```

SLAM with TurtleBot 3 in Gazebo

Terminal 3:

```
source /opt/ros/humble/setup.bash
export ROS_LOCALHOST_ONLY=1
export TURTLEBOT3_MODEL=burger
ros2 launch turtlebot3_cartographer \
  cartographer.launch.py use_sim_time:=True
```

Save map

Terminal 4:

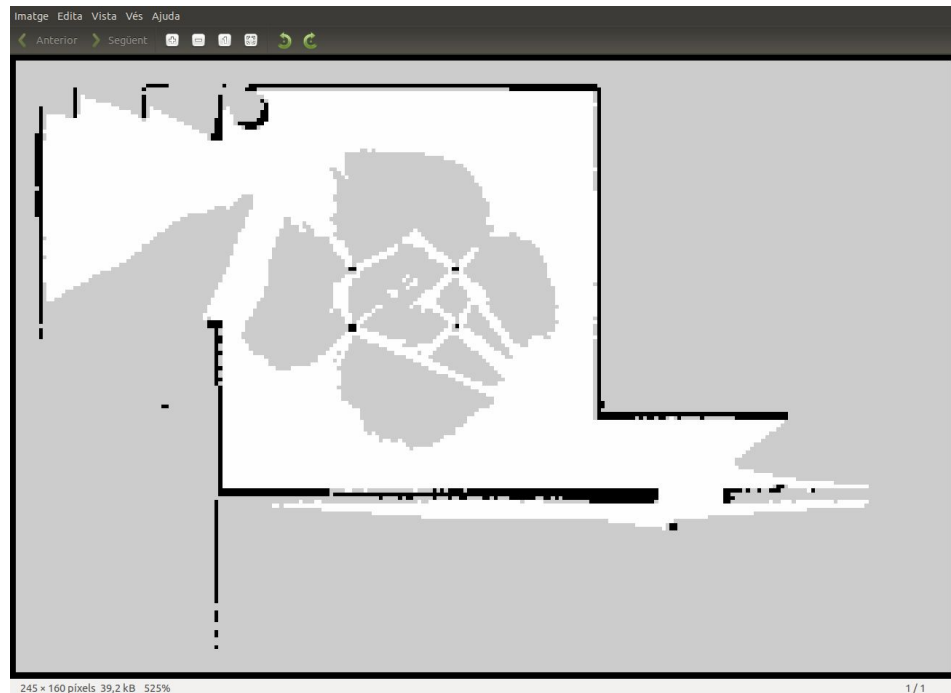
```
source /opt/ros/humble/setup.bash
export ROS_LOCALHOST_ONLY=1
ros2 run nav2_map_server map_saver_cli -f ./map
```

Save map

map.yaml

```
image: ./map.pgm
mode: trinary
resolution: 0.05
origin: [-5.65, -2.02, 0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.25
```

map.pgm

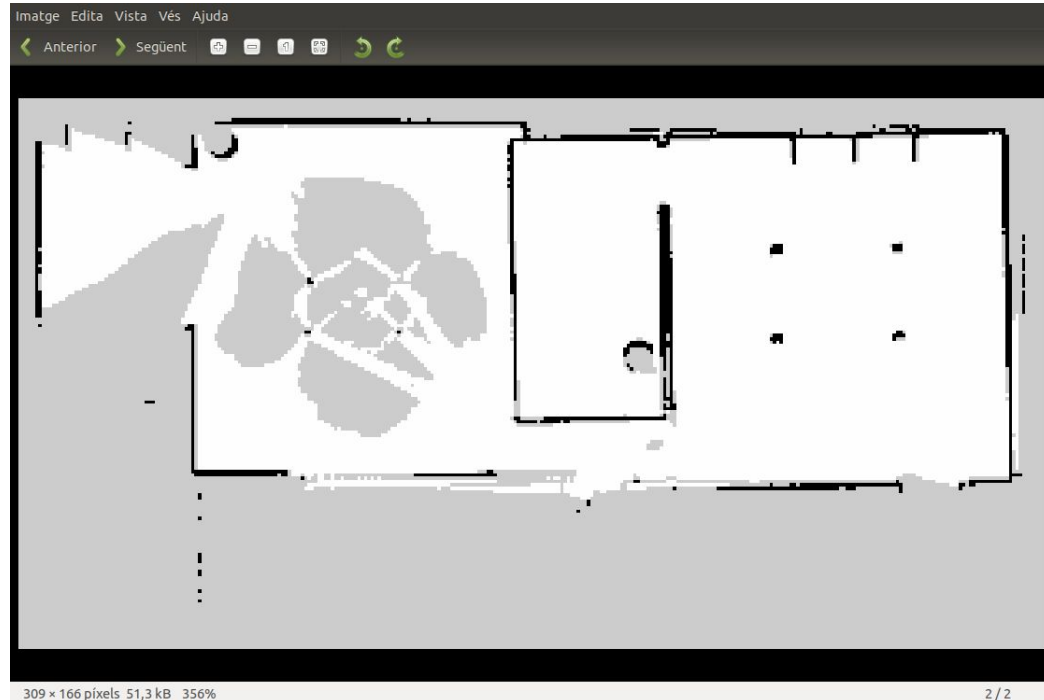


Save new map

map2.yaml

```
image: ./map2.pgm
mode: trinary
resolution: 0.05
origin: [-5.74, -2.44, 0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.25
```

map2.pgm

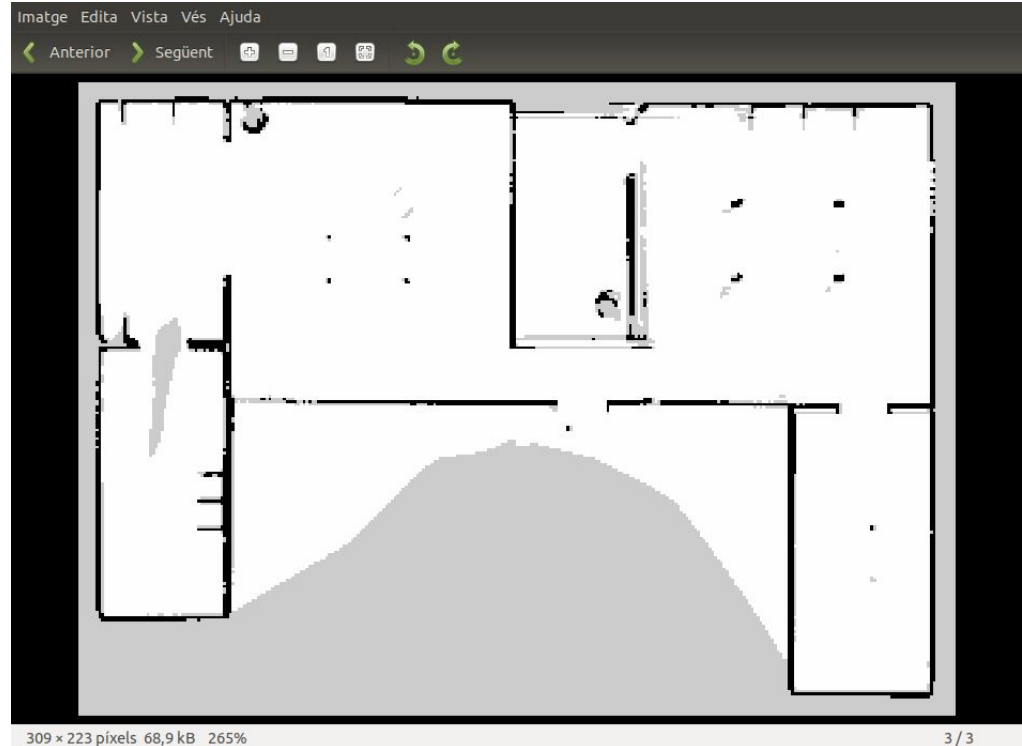


Save new map

map3.yaml

```
image: ./map3.pgm
mode: trinary
resolution: 0.05
origin: [-5.68, -5.2, 0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.25
```

map3.pgm



SLAM in ROS2: Cartographer

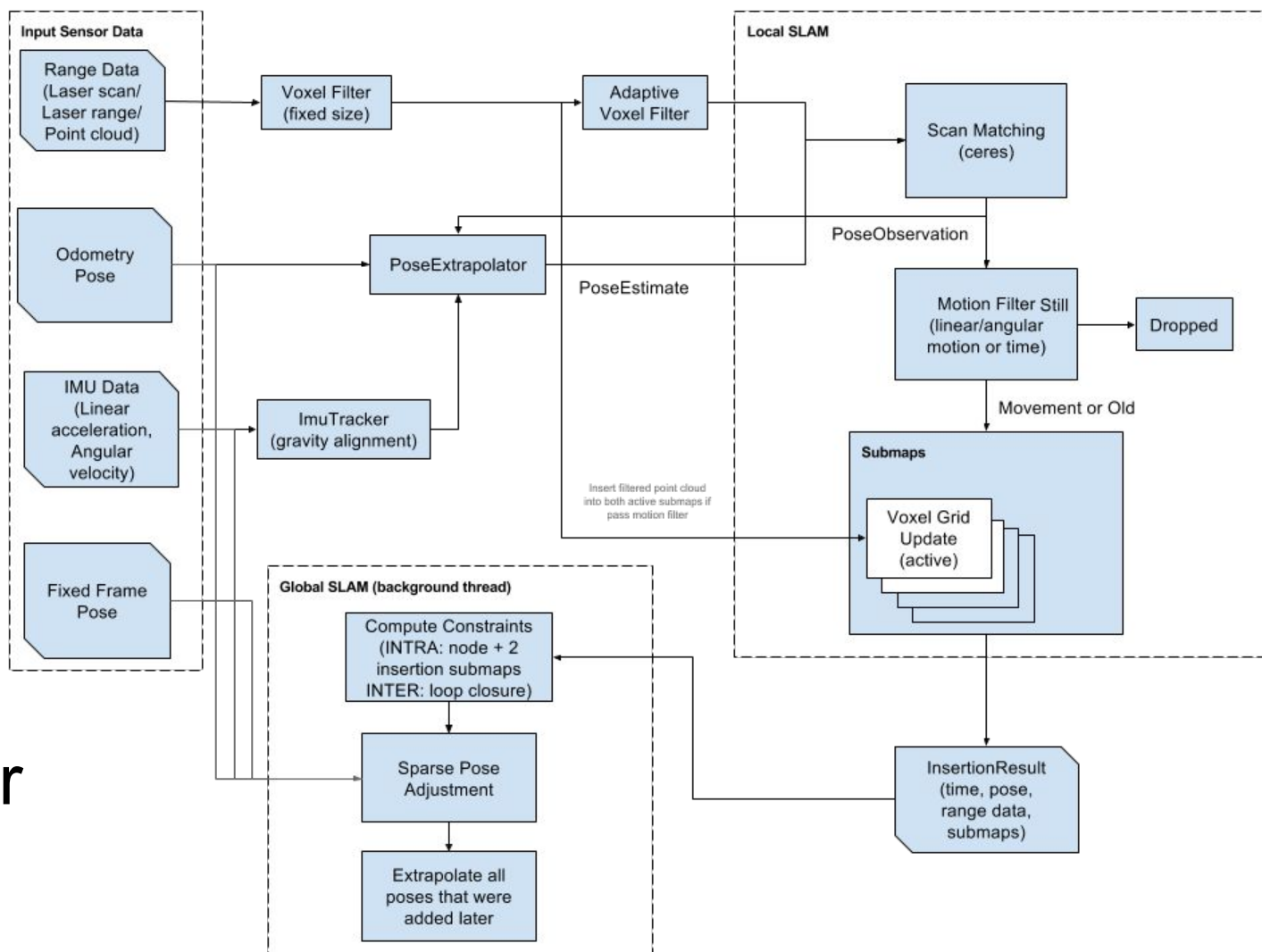
Cartographer is a complex system and tuning it requires a good understanding of its inner working.

If you are interested in more than an introduction to Cartographer, you should refer to the Cartographer paper (below).

It describes the 2D SLAM and defines rigourously most of the concepts described here. Those concepts generally apply to 3D as well.

W. Hess, D. Kohler, H. Rapp, and D. Andor, [Real-Time Loop Closure in 2D LIDAR SLAM](#), in Robotics and Automation (ICRA), 2016 IEEE International Conference on. IEEE, 2016. pp. 1271–1278.

Cartographer Overview



Overview 1

Cartographer can be seen as two separate, but related subsystems.

The first one is local SLAM
(sometimes also called frontend or local trajectory builder).
Its job is to build a succession of submaps.

Each submap is meant to be locally consistent
but we accept that local SLAM drifts over time.

Overview 2

The other subsystem is global SLAM (sometimes called the backend).

It runs in background threads and its main job is to find loop closure constraints.

It does that by scan-matching scans (gathered in nodes) against submaps.

It also incorporates other sensor data to get a higher level view and identify the most consistent global solution.

Input

Range finding sensors (e.g. LIDARs) provide **depth information in multiple directions**. However, some of the measurements are irrelevant for SLAM.

If the sensor is partially covered with dust or if it is directed towards a part of the robot, some of the measured distance can be considered as noise for SLAM.

On the other hand, some of the furthest measurements can also come from undesired sources (reflection, sensor noise) and are irrelevant for SLAM as well.

To tackle those issue, Cartographer starts by applying a bandpass filter and only **keeps range values between a certain min and max range**.

Those min and max values should be chosen according to the specifications of your robot and sensors.

Local SLAM

Once a scan has been assembled and filtered from multiple range data, it is ready for the local SLAM algorithm.

Local SLAM inserts a new scan into its current submap construction by scan matching using an initial guess from the pose extrapolator.

The idea behind the pose extrapolator is to use sensor data of other sensors besides the range finder to predict where the next scan should be inserted into the submap.

Global SLAM

While the local SLAM generates its succession of submaps, a global optimization task runs in background.

Its role is to re-arrange submaps between each other so that they form a coherent global map.

For instance, this optimization is in charge of altering the currently built trajectory to properly align submaps with regards to loop closures.

Task

Teleoperate the robot in the Turtlebot 3 Gazebo house environment

Run Cartographer SLAM

Record the following topics in a ROS bag:

```
/clock /map /map_updates /odom /robot_description /scan  
/scan_matched_points2 /submap_list /tf /tf_static
```

During teleoperation, save a sequence of maps with the map saver utility

Create a PDF document with the sequence of map images

Submit the (zipped) ROS bag file and the PDF document to Aula Virtual

You can verify your bag file with:

```
rviz2 -d /opt/ros/humble/share/turtlebot3_cartographer/rviz/tb3_cartographer.rviz
```

3D Model Editor

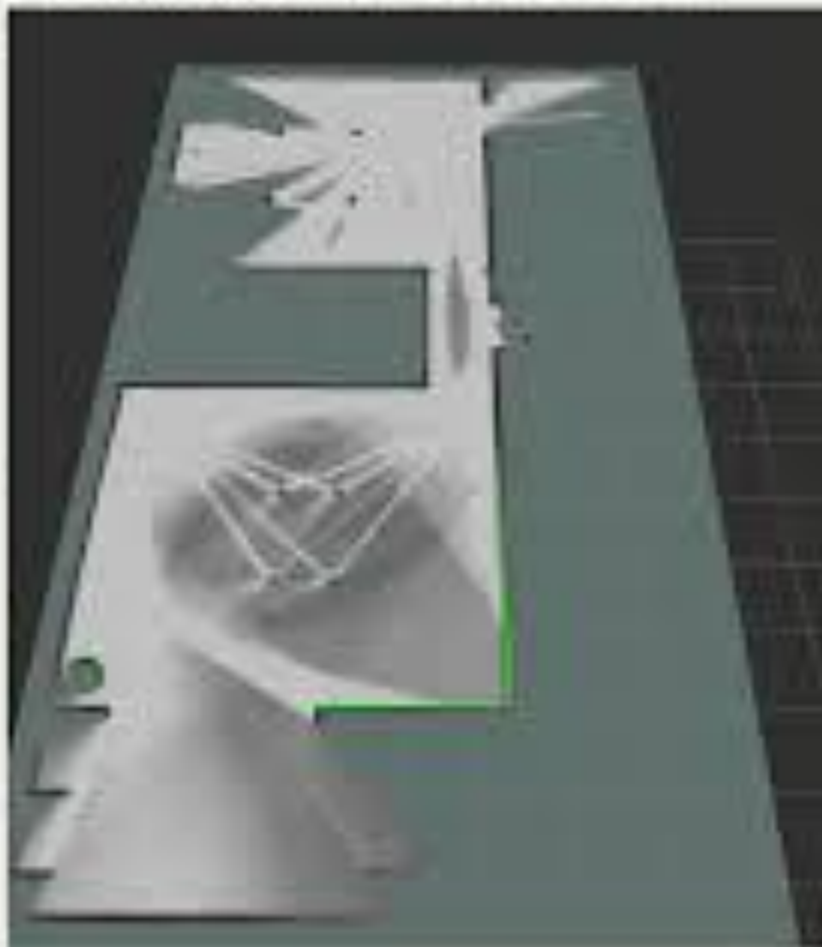
- 3D Model Editor
 - Global Options
 - Fixed Frame: map
 - Background Color: 15, 15, 15
 - Frame Rate: 30
 - Global Station Obj
 - Grid: ☒
 - IT: ☒
 - Station Obj
 - Show Holes: ☒
 - Show Jaws: ☒
 - Show Arrows: ☒
 - Marker Size: 1
 - Update Interval: 0
 - Frame Timeout: 11
 - Plotted: ☒
 - Time: ☒
 - Water/Cloud: ☒
 - Geometry: ☐
 - Map: ☒
 - Navigation: ☐
 - Command Panel
 - Isa, RAS, etc: ☒
 - Isa Inspector: ☐
 - Isa Constraints: ☐
 - Isa Landmark Views: ☐

Setup

OK

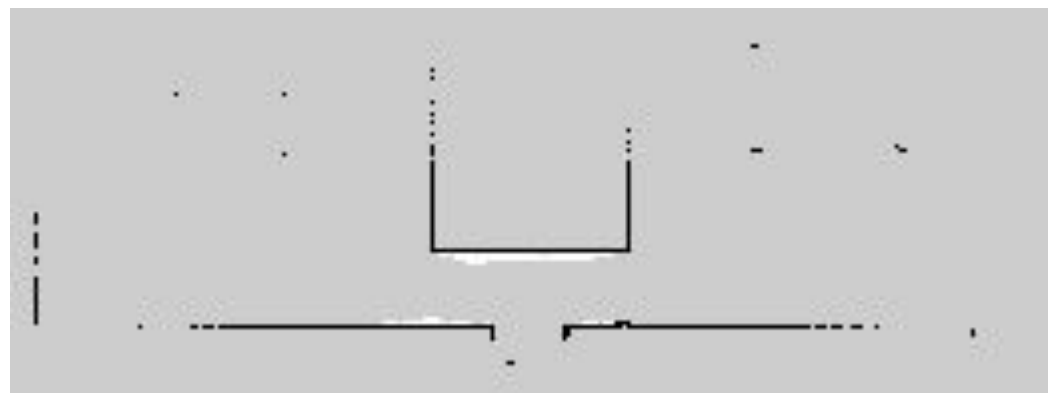
Add | Load | Save | Print | Close

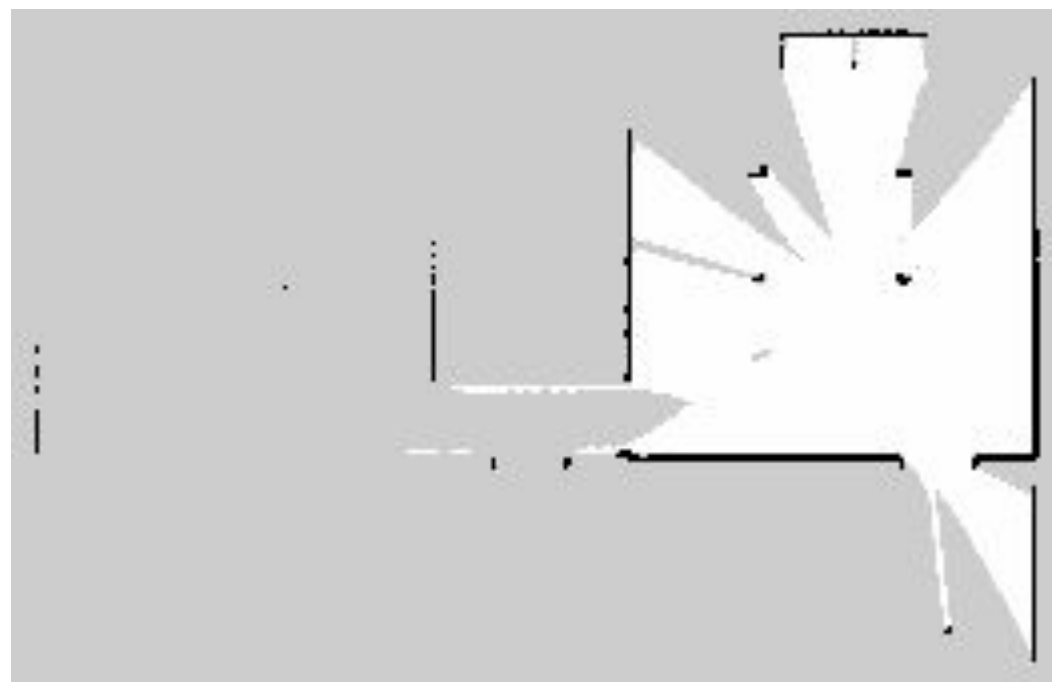
Setup

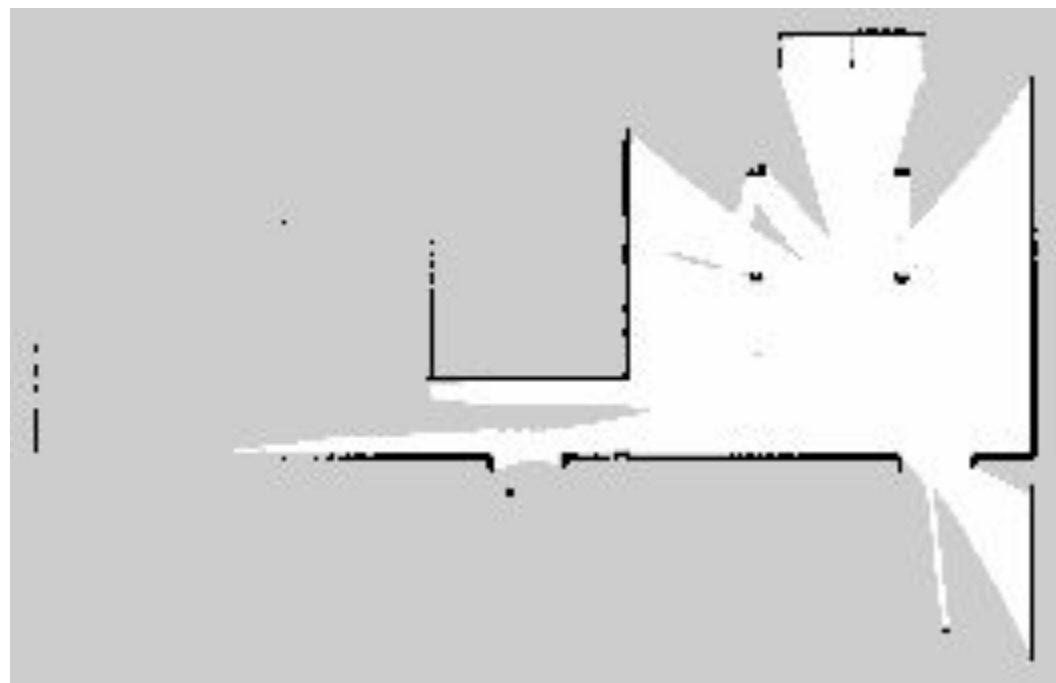


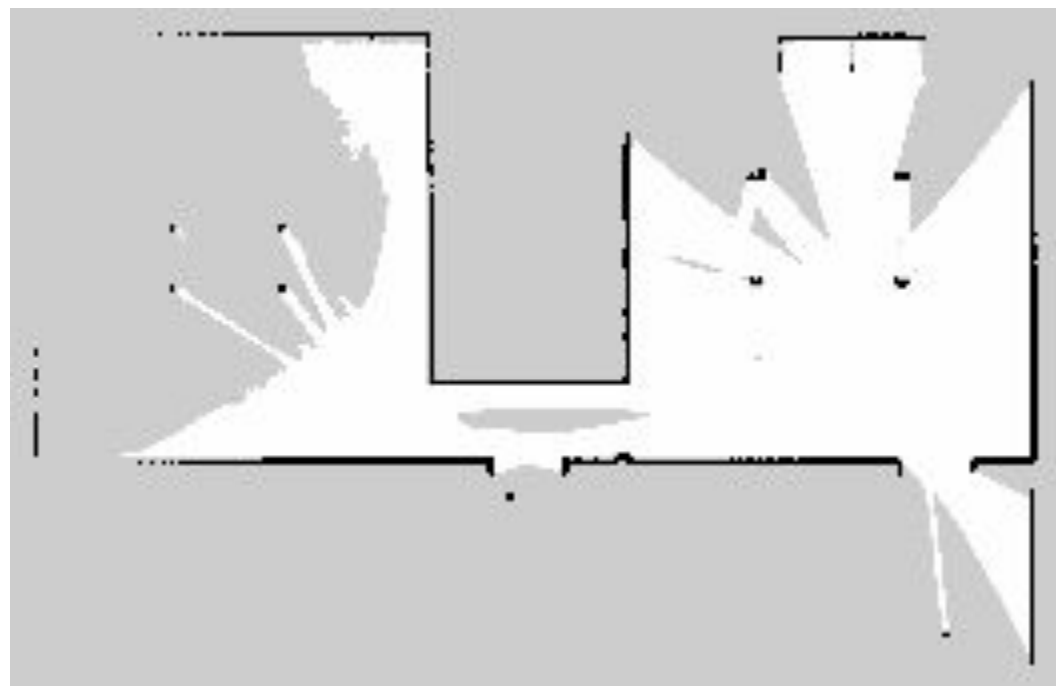


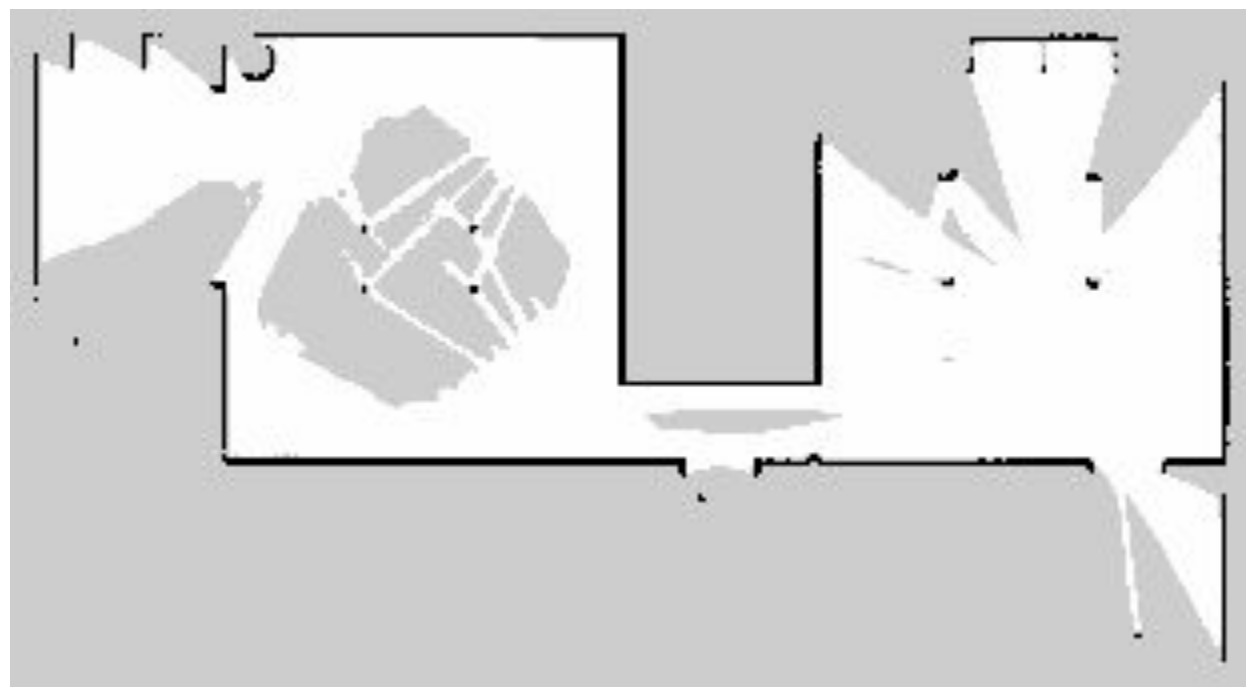




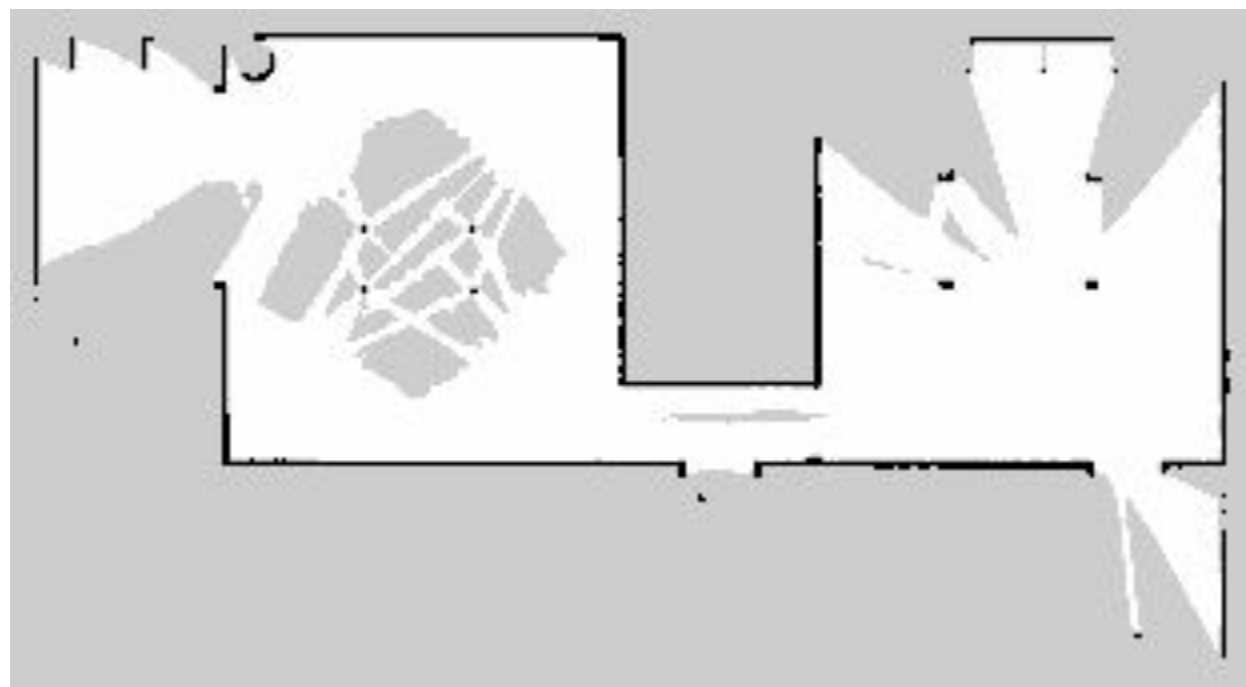


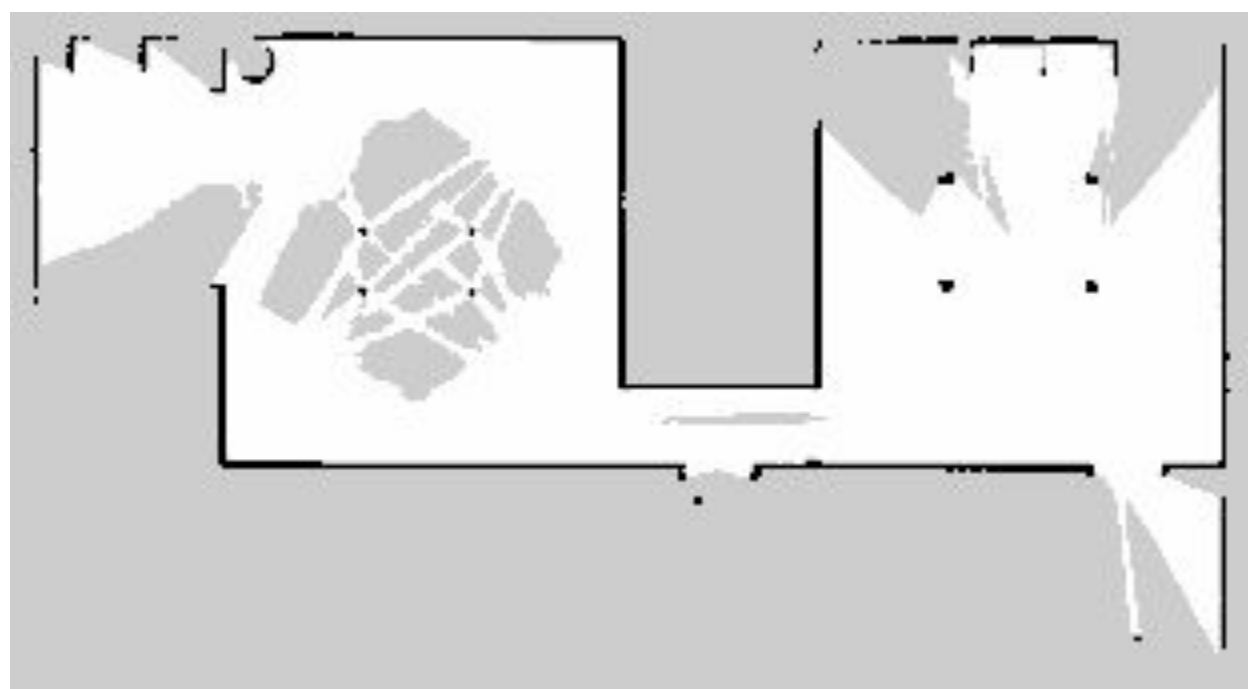












Task

Teleoperate the real Turtlebot 3 robot in the laboratory building

Run Cartographer SLAM

Record the following topics in a ROS bag:

```
/map /map_updates /odom /robot_description /scan  
/scan_matched_points2 /submap_list /tf /tf_static
```

Save a sequence of maps with the map saver utility

Create a PDF document with the sequence of map images

Overlap the last image with the map of the TD building

Submit the ROS bag file and the PDF document to Aula Virtual

Displays

Global Options

Fixed Frame	map
Background Color	48; 48; 48
Frame Rate	30
Global Status: Ok	
Grid	☒
TF	☒
LaserScan	☒
Status: Ok	
Topic	/scan
Selectable	☒
Style	Points
Size (Pixels)	3
Alpha	1
Decay Time	0
Position Transformer	XYZ
Color Transformer	Intensity
Channel Name	intensity
Use rainbow	☒
Invert Rainbow	☐
Autocompute Intens...	☒
Odometry	☐
Map	☒
Navigation	☐
Cartographer	☒
scan_matched_...	☒
Status: Ok	
Topic	/scan_matched_p...
Selectable	☒
Style	Points
Size (Pixels)	3
Alpha	1
Decay Time	0
Position Transfor...	XYZ
Color Transformer	FlatColor
Color	0; 255; 0
Trajectories	☐

Fixed Frame

Frame into which all data is transformed before being displayed.

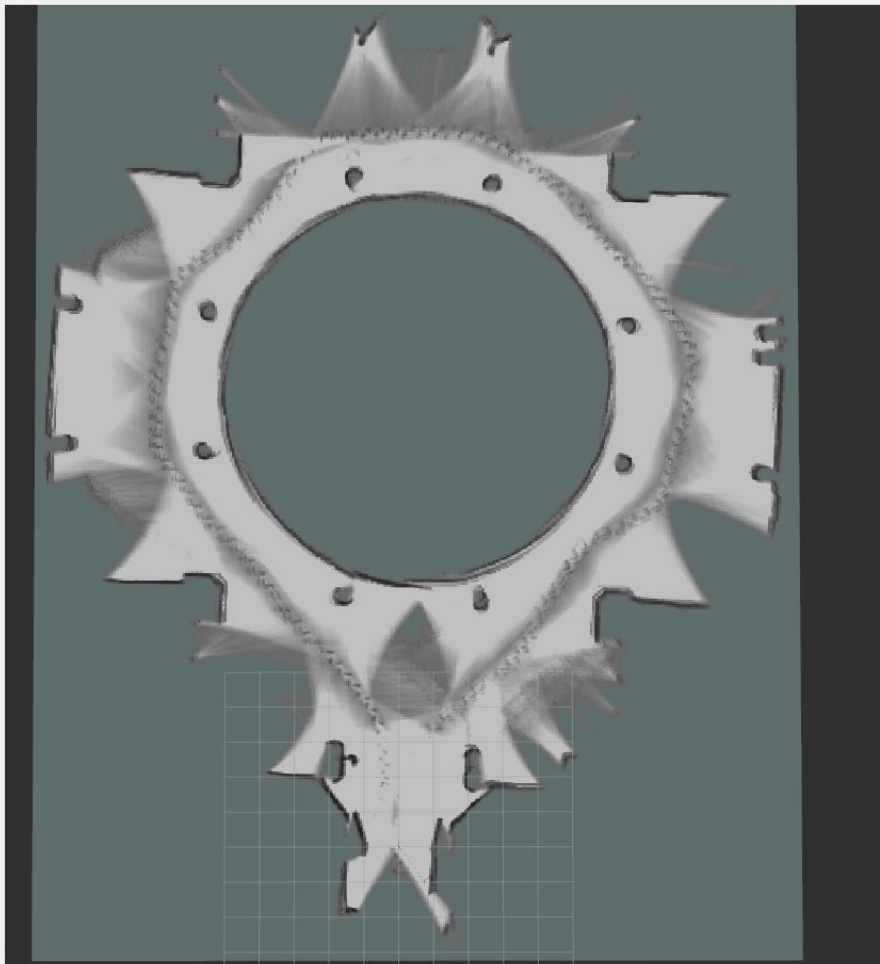
Add

Duplicate

Remove

Rename

Reset



Displays

Global Options

Fixed Frame map
 Background Color 48; 48; 48
 Frame Rate 30

Global Status: Ok

Grid

TF

LaserScan

Status: Ok

Topic

Selectable

Style

Size (Pixels)

Alpha

Decay Time

Position Transformer

Color Transformer

Channel Name

Use rainbow

Invert Rainbow

Autocompute Intens...

Odometry

Map

Navigation

Cartographer

scan_matched_...

Status: Ok

Topic

Selectable

Style

Size (Pixels)

Alpha

Decay Time

Position Transfor...

Color Transformer

Color

Trajectories

Fixed Frame

Frame into which all data is transformed before being displayed.

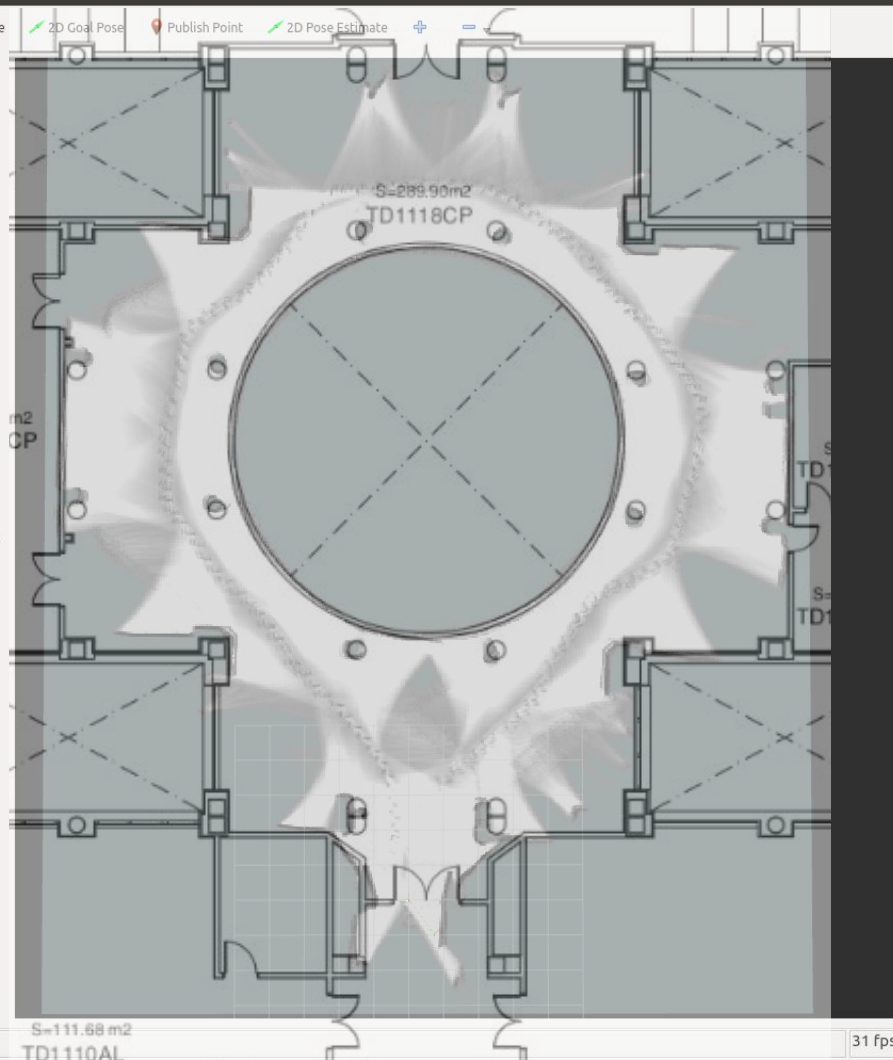
Add

Duplicate

Remove

Rename

Reset



Other SLAM algorithms

ROS2 SLAM Toolbox

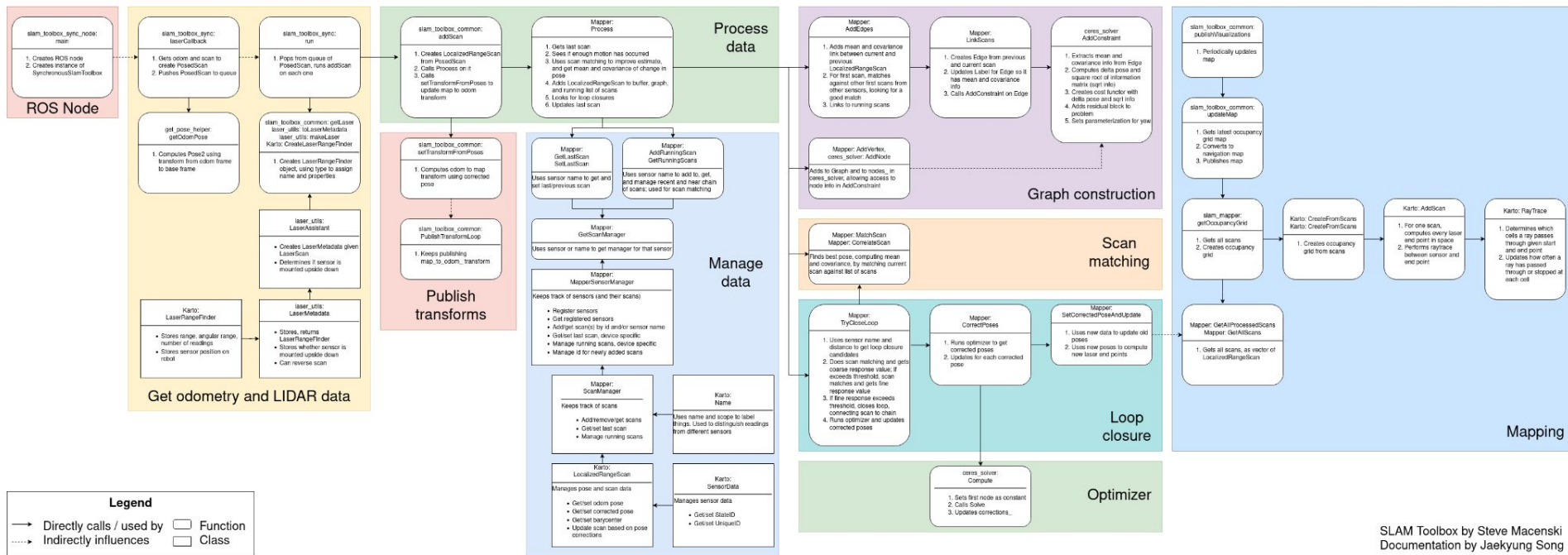
https://github.com/SteveMacenski/slam_toolbox

RTAB-Map

[rtabmap_ros/rtabmap](https://github.com/RTAB-Map/rtabmap_ros)

ROS2 SLAM Toolbox

SLAM TOOLBOX



SLAM Toolbox

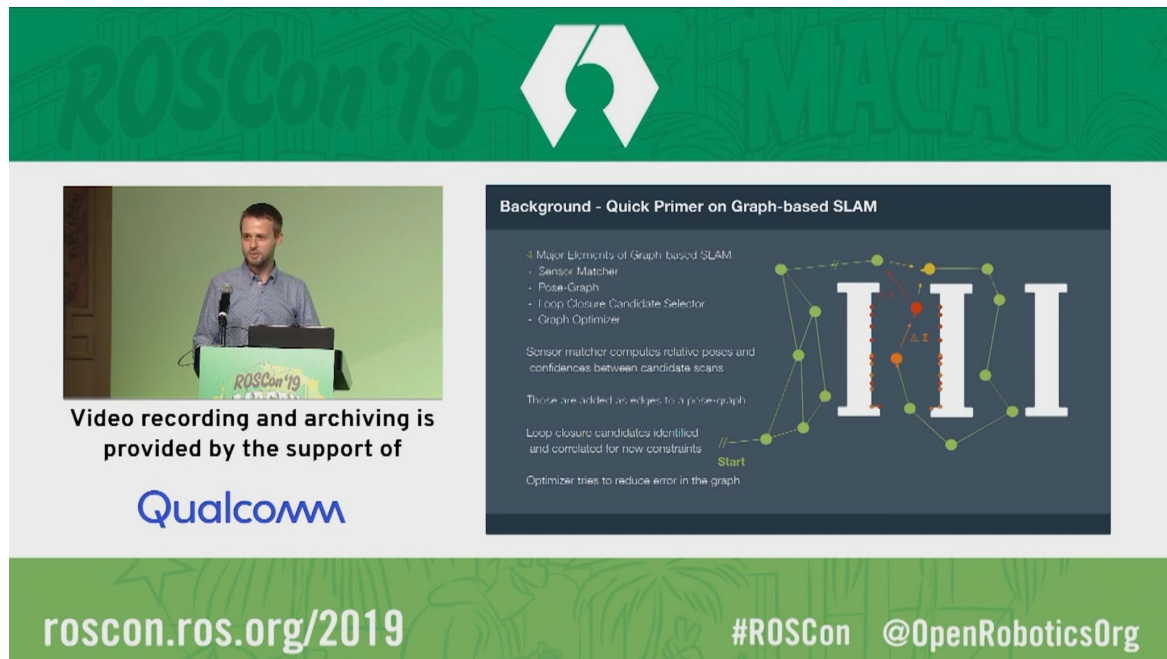
Install:

```
sudo apt install ros-humble-slam-toolbox
```

Run:

```
ros2 launch slam_toolbox \  
  online_async_launch.py \  
  use_sim_time:=True
```

SLAM Toolbox



The slide features a green header with the ROSCon'19 logo and the word 'MACAU'. Below the header, on the left, is a video frame of a speaker at a podium with a 'ROSCon'19' sign. To the right of the video is a diagram titled 'Background - Quick Primer on Graph-based SLAM'. The diagram lists four major elements: Sensor Matcher, Pose Graph, Loop Closure Candidate Selector, and Graph Optimizer. It includes a visual representation of a pose graph with nodes and edges, a loop closure candidate identified and correlated for new constraints, and a graph optimizer that tries to reduce error. The bottom of the slide has a green footer with the URL 'roscon.ros.org/2019', the hashtag '#ROSCon', and the Twitter handle '@OpenRoboticsOrg'.

ROSCon'19 MACAU

Video recording and archiving is provided by the support of

Qualcomm

Background - Quick Primer on Graph-based SLAM

- 4 Major Elements of Graph-based SLAM
 - Sensor Matcher
 - Pose Graph
 - Loop Closure Candidate Selector
 - Graph Optimizer

Sensor matcher computes relative poses and confidences between candidate scans.

These are added as edges to a pose-graph.

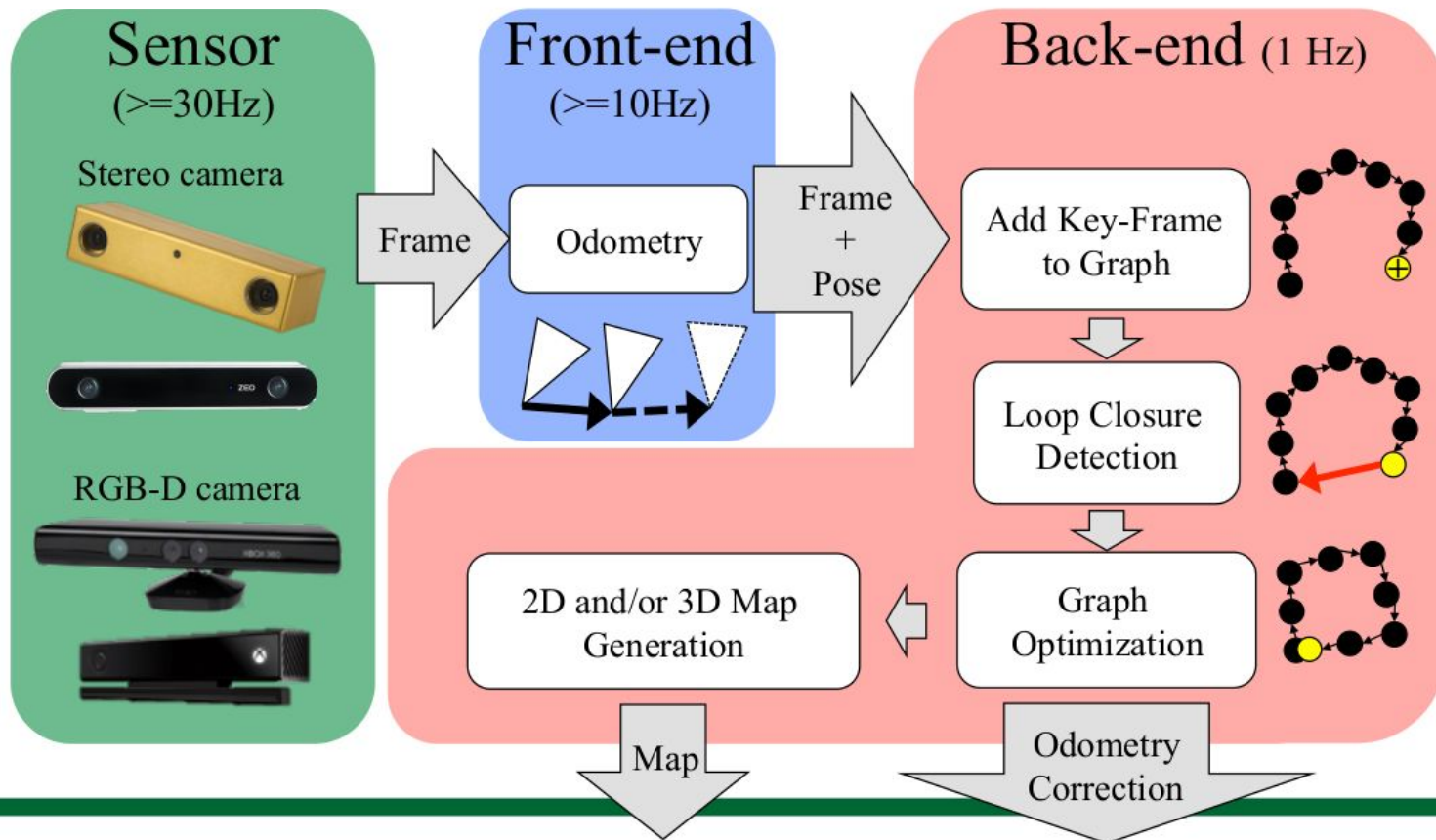
Loop closure candidates identified and correlated for new constraints

Optimizer tries to reduce error in the graph

roscon.ros.org/2019 #ROSCon @OpenRoboticsOrg

Macenski, S., Jambrecic I., SLAM Toolbox: [SLAM for the dynamic world](#), *Journal of Open Source Software*, 6(61), 2783, 2021.

RTAB-Map: Real-Time Appearance-Based Mapping



RTAB-Map

Install:

```
sudo apt install ros-humble-rtabmap-ros
```

Run:

```
ros2 launch rtabmap_demos \  
  turtlebot3_scan.launch.py \  
  use_sim_time:=True
```

RTAB-Map



M. Labbé and F. Michaud, “[RTAB-Map as an Open-Source Lidar and Visual SLAM Library for Large-Scale and Long-Term Online Operation](#),” in *Journal of Field Robotics*, vol. 36, no. 2, pp. 416–446, 2019.

HOMEWORK: Comparison of SLAM methods

Teleoperate the robot in your Gazebo model of the TD building.

Record the following topics in a ROS bag:

```
/clock /odom /robot_description /scan /tf /tf_static
```

Run each SLAM algorithm (Cartographer, SLAM Toolbox, RTAB-Map) on a play of the bag file and save a sequence of the generated maps.

Create a PDF document with the sequence of map images, and add your comments about the result of each algorithm.

Submit the (zipped) ROS bag file and the PDF document.